RSACryptoOAEP 加密库

RSA 加密算法介绍

一、背景

RSA 公钥加密算法是 1977 年由罗纳德·李维斯特（Ron Rivest）、阿迪·萨莫尔（Adi Shamir）和伦纳德·阿德曼（Leonard Adleman）一起提出的，1987 年首次公布。

RSA 是目前最有影响力的公钥加密算法，它能够抵抗到目前为止已知的绝大多数密码攻击，已被 ISO 推荐为公钥数据加密标准。

今天只有短的 RSA 钥匙才可能被强力方式解破。到 2008 年为止，世界上还没有任何可靠的攻击 RSA 算法的方式。只要其钥匙的长度足够长，用 RSA 加密的信息实际上是不能被解破的。但在分布式计算和量子计算机理论日趋成熟的今天，RSA 加密安全性受到了挑战。

RSA 算法基于一个十分简单的数论事实：将两个大质数相乘十分容易，但是想要对其乘积进行因式分解却极其困难，因此可以将乘积公开作为加密密钥。

RSA 算法是现今使用最广泛的公钥密码算法，也是号称地球上最安全的加密算法。在了解 RSA 算法之前，先熟悉下几个术语

根据密钥的使用方法，可以将密码分为对称密码和公钥密码

- 对称密码：加密和解密使用同一种密钥的方式
- 公钥密码：加密和解密使用不同的密码的方式，因此公钥密码通常也称为非对称密码。

二、RSA 算法描述

（一）密钥计算方法

1. 选择两个大素数 p 和 q(典型值为 1024 位)
2. 计算 n=p×q 和 z=(p-1)×(q-1)（n 表示欧拉函数）
3. 选择一个与 z 互质的数，令其为 d
4. 找到一个 e 使满足 exd= 1 (mod z)
5. 公开密钥为(e，m)，私有密钥为(d，m)

（二）加密方法

1. 将明文看成比特串，将明文划分成 k 位的块 P 即可，这里 k 是满足 2*k<n 的最大整数。
2. 对每个数据块 P，计算 C= P^(mod n),C 即为 P 的密文。

加密

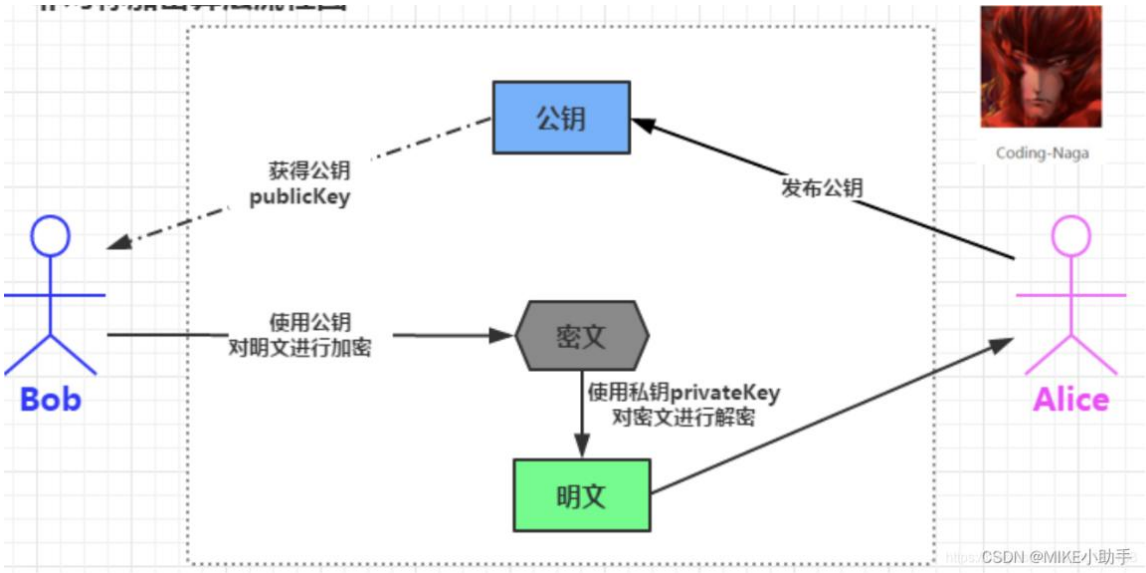
$$C = P^{e}(\mathrm{mod}n)$$

（三）解密方法

对每个密文块 C，计算 P=C^d(mod n),P 即为明文

解密：

$$P = c^{d}(\mathrm{mod}n)$$



RSA 算法流程图

最优非对称加密填充(OAEP)

在密码学中，最优非对称加密填充（英语：Optimal Asymmetric Encryption Padding，缩写：OAEP）是一种经常与 RSA 加密一起使用的填充方案。OAEP 由 Mihir Bellare 和 Phillip Rogaway 发明，随后在 PKCS#1 v2 和 RFC 2437 中得到标准化。

OAEP 算法是费斯妥密码的一种形式，它使用一对随机预言 G 和 H 在进行非对称加密之前处理明文。OAEP 与任何安全的陷门单向置换 f 结合使用在随机预言模型中被证明是一种在选择明文攻击（IND-CPA）下语义安全的组合方案。当使用某些陷门置换（如 RSA）实现时，OAEP 也被证明可以抵抗选择密文攻击。OAEP 可用于构建全有或全无转换（all-or-nothing transform）。

OAEP 满足以下两个目标：

1. 添加随机性元素，这可用于将确定性加密方案（如传统 RSA）转变为概率加密方案。
2. 通过确保无法反转陷门单向置换 f，从而无法恢复明文的任何部分，来防止密文的部分解密（或造成其他信息泄漏）。

当 OAEP 与任何陷门置换一起使用时，OAEP 的原始版本（Bellare/Rogaway, 1994）在随机预言机模型中显示了一种“明文知晓性”的形式（他们声称这意味着对选择密文攻击是安全的）。然而随后的结果与这一点相抵触，表明 OAEP 仅是 IND-CCA1 安全的。但是与 RSA-OAEP 的情况一样，当将 OAEP 与使用标准加密指数的 RSA 置换一起使用时，在随机预言模型中证明了原始方案是 IND-CCA2 安全的。Victor Shoup 提供了一种改进的方案（称为 OAEP+），该方案可与任何陷门单向置换配合使用，以解决此问题。近期的研究表明，在标准模型中（即当哈希函数未建模为随机预言时），无法在假定 RSA 问题的难度下证明 RSA-OAEP 具有 IND-CCA2 安全性。

RSACryptoOAEP 库

保密原理

1. 利用开源的 OpenSSL 算法库实现 RSA 和 OAEP 算法。根据克克霍夫原则：
   一个加密系统应该在敌人知道其加密算法的前提下依然是安全的。
   即使使用开源的库，RSACryptoOAEP 依然能保证安全。
2. 生成的密钥长度为 2048 位且随机，难以通过公钥破解私钥。没有私钥则不能解密。
3. 通过运用 OAEP 算法，实现了对一些难以保证不同的信息，比如时间，的加密，且相同的明文可以产生不同的密文，降低了破解的可能。

使用说明

RSACryptoOAEP();
默认构造函数，在程序文件夹下寻找 public.pem（公钥文件）和 private.pem（私钥文件），若找不到则自行生成。将公私钥文件初始化。

RSACryptoOAEP(std::string PuPP, std::string PrPP);
PuPP：公钥文件地址。
PrPP：私钥文件地址。
含参构造函数，在参数文件夹下寻找公钥文件和私钥文件，若找不到则自行生成。将公私钥指针初始化。

~RSACryptoOAEP();
析构函数，释放指针。

```
void set_key_path(std::string pupp, std::string prpp);

/**
 * 生成 RSA 密钥对并写入 PEM 文件
 *
 * @param 2048          密钥长度
 * @param privPath      私钥文件路径（PEM, PKCS#8）
 * @param pubPath       公钥文件路径（PEM, SubjectPublicKeyInfo）
 * @param passphrase    给私钥加密码（可为 nullptr 表示不加密）
 */
void generateKeyPair(const std::string& privPath,
    const std::string& pubPath,
    const char* passphrase = nullptr);
```

privPath：私钥生成地址。
pubPath：公钥生成地址。
passphrase ：私钥密码。
密钥生成函数，在指定位置生成公私钥，可选是否对密钥加密。

```
/**
 * 生成 RSA 密钥对并写入 PEM 文件
 *
 * @param 2048          密钥长度
 * @param passphrase    给私钥加密码（可为 nullptr 表示不加密）
 */
void generateKeyPair(const char* passphrase = nullptr);
```

passphrase ：私钥密码。
密钥生成函数，在默认位置（当前文件夹下的 public.pem 和 private.pem）生成公私钥，可选是否对密钥加密。

```
std::vector<unsigned char> encrypt(const std::vector<unsigned char>& plain);
```

Plain：明文。
返回值：密文。
加密函数，重载为 std::vector<unsigned char>型参数。

```
std::vector<unsigned char> encrypt(const std::string& plain);
```

Plain：明文。
返回值：密文。
加密函数，重载为 std::string 型参数。

```
std::vector<unsigned char> decrypt(const std::vector<unsigned char>& cipher);
```

Cipher：密文。
返回值：明文。
解密函数，重载为 std::vector<unsigned char>型参数。

```
std::vector<unsigned char> decrypt(const std::string& cipher);
```

Cipher：密文。
返回值：明文。
解密函数，重载为 std::string 型参数。

```
void print_plain(std::vector<unsigned char>& cipher);
```

Cipher：密文。
密文打印函数，将密文转为合适编码，cout 到控制台。

```
void print_cipher(std::vector<unsigned char>& plain);
```

Plain：明文。

明文打印函数，将明文转为合适编码，cout 到控制台。